

Zeros de Funções: Método da Bisseção e Método da Falsa Posição

Algoritmos, implementação em Python e aplicações em Ciência e Tecnologia

Prof. Paulo César Linhares da Silva

EXA1132 – Cálculo Numérico
Universidade Federal Rural do Semi-Árido – UFERSA

2026.2

- 1 Introdução
- 2 Método da Bisseção
- 3 Método da Falsa Posição
- 4 Problemas aplicados
- 5 Aplicações para alunos de Ciência da Computação – UFERSA
- 6 Síntese

O problema de encontrar zeros de funções

Problema

Dada uma função $f(x)$ contínua, encontrar x^* tal que

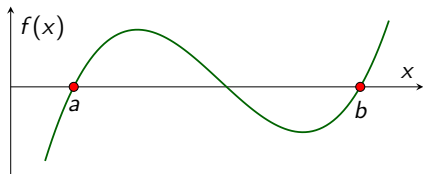
$$f(x^*) = 0$$

- Muitas equações não possuem solução analítica fechada (ex.: $x = \cos x$, $e^{-x} - x = 0$).
- Métodos **numéricos iterativos** constroem uma sequência $x_0, x_1, x_2, \dots$ que converge para a raiz x^* .
- Hoje veremos dois métodos **de intervalo** (bracketing methods): sempre mantêm a raiz confinada em um intervalo, garantindo convergência.
 - Método da **Bisseção**
 - Método da **Falsa Posição** (*Regula Falsi*)

Teorema de Bolzano (condição de existência)

Teorema do Valor Intermediário (Bolzano)

Se f é contínua em $[a, b]$ e $f(a) \cdot f(b) < 0$ (sinais opostos), então existe pelo menos um $x^* \in (a, b)$ tal que $f(x^*) = 0$.

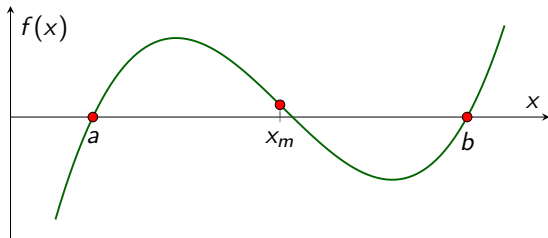


Ambos os métodos de hoje exploram exatamente essa ideia: um intervalo $[a, b]$ com $f(a) \cdot f(b) < 0$ garante que há raiz lá dentro.

Método da Bisseção – ideia

- Parte de um intervalo $[a, b]$ com $f(a) \cdot f(b) < 0$.
- Calcula o ponto **médio**: $x_m = \frac{a + b}{2}$.
- Verifica em qual metade a raiz permanece, comparando o sinal de $f(x_m)$ com o sinal de $f(a)$:
 - Se $f(a) \cdot f(x_m) < 0$, a raiz está em $[a, x_m] \Rightarrow$ atualiza $b = x_m$;
 - Caso contrário, a raiz está em $[x_m, b] \Rightarrow$ atualiza $a = x_m$.
- Repete até que o intervalo seja suficientemente pequeno (critério de parada).
- Ideia central: **dividir o intervalo ao meio a cada iteração** – reduz a incerteza pela metade sempre.

Método da Bisseção – interpretação geométrica



x_m é sempre o **ponto médio geométrico** do intervalo, independentemente do formato de f .

Bisseção – algoritmo (pseudocódigo)

```
1 função BISSECAO(f, a, b, tol, max_iter)
2   se f(a) * f(b) >= 0 então
3     erro "f(a) e f(b) devem ter sinais opostos"
4   fim se
5   para i = 1 até max_iter faça
6     xm = (a + b) / 2
7     se |f(xm)| < tol ou (b - a)/2 < tol então
8       retorne xm                                # convergiu
9     fim se
10    se f(a) * f(xm) < 0 então
11      b = xm                                     # raiz na metade esquerda
12    senão
13      a = xm                                     # raiz na metade direita
14    fim se
15  fim para
16  retorne (a + b) / 2                            # aproximacao apos max_iter
17 fim função
```

Bisseção – código em Python

```
1 def bissecao(f, a, b, tol=1e-6, max_iter=100):
2     """Metodo da Bissecao para encontrar raiz de f em [a, b]."""
3     if f(a) * f(b) >= 0:
4         raise ValueError("f(a) e f(b) devem ter sinais opostos")
5
6     for i in range(1, max_iter + 1):
7         xm = (a + b) / 2
8         fxm = f(xm)
9         if abs(fxm) < tol or (b - a) / 2 < tol:
10             return xm, i          # raiz aproximada, num. iteracoes
11         if f(a) * fxm < 0:
12             b = xm
13         else:
14             a = xm
15     return (a + b) / 2, max_iter
16
17 # Exemplo: raiz de f(x) = x**3 - x - 2 em [1, 2]
18 raiz, n_iter = bissecao(lambda x: x**3 - x - 2, 1, 2)
19 print(f"raiz = {raiz:.6f} (em {n_iter} iteracoes)")
```


Bisseção – exemplo passo a passo

Encontrar a raiz de $f(x) = x^3 - x - 2$ em $[1, 2]$ (note $f(1) = -2 < 0$ e $f(2) = 4 > 0$).

Iter.	a	b	x_m	$f(x_m)$	Novo intervalo
1	1,0000	2,0000	1,5000	+0,8750	[1,0000, 1,5000]
2	1,0000	1,5000	1,2500	-1,2969	[1,2500, 1,5000]
3	1,2500	1,5000	1,3750	-0,2246	[1,3750, 1,5000]
4	1,3750	1,5000	1,4375	+0,3062	[1,3750, 1,4375]
5	1,3750	1,4375	1,4063	+0,0345	[1,3750, 1,4063]

A cada iteração o intervalo $[a, b]$ é reduzido pela metade, aproximando-se de $x^* \approx 1,3960$.

Bisseção – convergência e número de iterações

- A cada iteração, o comprimento do intervalo é dividido por 2:

$$|b_n - a_n| = \frac{b_0 - a_0}{2^n}$$

- Para garantir um erro menor que uma tolerância ε , basta resolver:

$$\frac{b_0 - a_0}{2^n} < \varepsilon \Rightarrow n > \log_2 \left(\frac{b_0 - a_0}{\varepsilon} \right)$$

Vantagem e desvantagem

Vantagem: convergência **garantida** desde que f seja contínua e $f(a) \cdot f(b) < 0$ – número de iterações pode ser **calculado a priori**.

Desvantagem: convergência **linear** (relativamente lenta) – não usa a informação do valor de f , apenas o sinal.

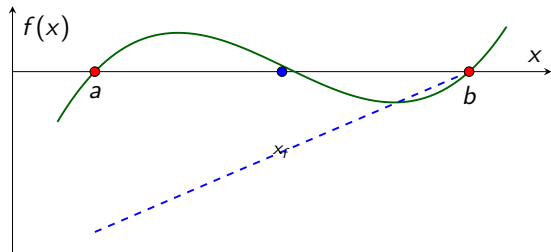
Método da Falsa Posição – ideia

- Também parte de $[a, b]$ com $f(a) \cdot f(b) < 0$, mas em vez do ponto médio, usa uma **interpolação linear** entre $(a, f(a))$ e $(b, f(b))$.
- O novo ponto x_f é onde a **reta secante** que liga esses dois pontos cruza o eixo x :

$$x_f = a - f(a) \cdot \frac{b - a}{f(b) - f(a)} = \frac{a f(b) - b f(a)}{f(b) - f(a)}$$

- Assim como na bisseção, verifica-se o sinal de $f(x_f)$ para decidir qual extremo atualizar.
- Ideia: usar o **valor** de f , não apenas o sinal – tende a convergir mais rápido quando f é "bem-comportada".

Falsa Posição – interpretação geométrica



A reta (secante, em azul) liga $(a, f(a))$ e $(b, f(b))$; x_f é onde ela cruza o eixo x – diferente do ponto médio geométrico x_m da bisseção.

Falsa Posição – algoritmo (pseudocódigo)

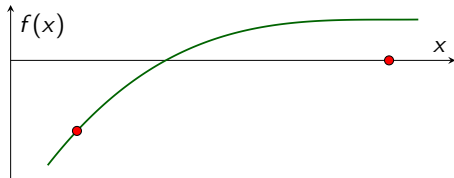
```
1 função FALSA_POSICAO(f, a, b, tol, max_iter)
2   se f(a) * f(b) >= 0 então
3     erro "f(a) e f(b) devem ter sinais opostos"
4   fim se
5   para i = 1 até max_iter faça
6     xf = (a * f(b) - b * f(a)) / (f(b) - f(a))
7     se |f(xf)| < tol então
8       retorne xf                                # convergiu
9     fim se
10    se f(a) * f(xf) < 0 então
11      b = xf                                     # raiz na metade esquerda
12    senão
13      a = xf                                     # raiz na metade direita
14    fim se
15  fim para
16  retorne xf
17 fim função
```

Falsa Posição – código em Python

```
1 def falsa_posicao(f, a, b, tol=1e-6, max_iter=100):
2     """Metodo da Falsa Posicao (Regula Falsi)."""
3     if f(a) * f(b) >= 0:
4         raise ValueError("f(a) e f(b) devem ter sinais opostos")
5
6     fa, fb = f(a), f(b)
7     for i in range(1, max_iter + 1):
8         xf = (a * fb - b * fa) / (fb - fa)
9         fxf = f(xf)
10        if abs(fxf) < tol:
11            return xf, i
12        if fa * fxf < 0:
13            b, fb = xf, fxf
14        else:
15            a, fa = xf, fxf
16    return xf, max_iter
17
18 # Exemplo: raiz de  $f(x) = x^3 - x - 2$  em  $[1, 2]$ 
19 raiz, n_iter = falsa_posicao(lambda x: x**3 - x - 2, 1, 2)
20 print(f"raiz = {raiz:.6f} (em {n_iter} iteracoes)")
```

Falsa Posição – o problema da estagnação

- Quando a função é **convexa** ou **côncava** perto da raiz, um dos extremos do intervalo pode **ficar parado** por várias iterações, enquanto o outro se aproxima lentamente da raiz.
- Resultado: convergência bem mais lenta que o esperado em alguns casos, apesar de usar mais informação que a bisseção.



Correção comum

Variante **Illinois**: quando um extremo fica parado, seu valor de f é dividido por 2 (ou outro fator) antes da próxima iteração, forçando o intervalo a se mover – recupera convergência mais rápida.

Bisseção vs. Falsa Posição – comparação

	Bisseção	Falsa Posição
Uso da informação de f	Apenas sinal	Sinal e valor
Novo ponto	Ponto médio	Interseção da secante
Convergência garantida	Sim	Sim
Velocidade típica	Linear, previsível	Pode ser mais rápida... ...ou estagnar em um lado
Número de iterações	Calculável a priori	Não facilmente previsível
Robustez	Muito alta	Alta, com ressalvas

- Ambos são métodos de **intervalo fechado**: nunca "escapam" da raiz, ao contrário de Newton-Raphson.
- Ambos exigem apenas que f seja **contínua** (não precisam de derivada).

Métodos de busca de raízes aparecem sempre que é preciso **calibrar um parâmetro** para que alguma grandeza atinja um valor-alvo – um problema extremamente comum em computação.

- Ajuste de parâmetros e limiares (*thresholds*);
- Dimensionamento de recursos computacionais;
- Modelagem de crescimento e desempenho de sistemas;
- Computação gráfica e simulação.

Nos próximos slides, veremos problemas concretos que os alunos de Ciência da Computação podem resolver com Bisseção ou Falsa Posição.

Problema 1 – Ponto de equilíbrio entre algoritmos

Enunciado

Um algoritmo A tem complexidade $T_A(n) = 50n$ e um algoritmo B tem complexidade $T_B(n) = 2n \log_2 n$. Encontre o valor de n (não inteiro, como aproximação contínua) a partir do qual B passa a ser mais eficiente que A .

- Formule como zero de função: $g(n) = T_A(n) - T_B(n) = 50n - 2n \log_2 n$.
- Encontrar n^* tal que $g(n^*) = 0$ define o **ponto de cruzamento** das duas curvas de desempenho.
- Relevante na prática: escolher, em tempo de execução, qual algoritmo usar dependendo do tamanho da entrada (estratégia usada em bibliotecas de ordenação híbridas, como o Timsort).

Problema 2 – Calibração de limiar em Machine Learning

Enunciado

Um classificador binário produz uma pontuação de confiança $s(x)$. Deseja-se encontrar o limiar t tal que a taxa de falsos positivos $FPR(t)$ seja exatamente igual a 5%.

- Formule como zero de função: $g(t) = FPR(t) - 0,05$.
- $FPR(t)$ é tipicamente **monótona decrescente** em t (limiar mais alto $\Rightarrow$ menos falsos positivos), garantindo troca de sinal.
- Bisseção é uma escolha natural, pois $FPR(t)$ pode não ter forma analítica fechada – é calculada a partir dos dados de validação.

Problema 3 – Interseção em computação gráfica (ray marching)

Enunciado

Em renderização por *ray marching*, um raio de luz é descrito por $P(t) = O + t D$ (origem O , direção D). A superfície de um objeto é definida implicitamente por $F(x, y, z) = 0$. Encontrar o ponto de interseção do raio com a superfície.

- Formule como zero de função unidimensional: $g(t) = F(P(t))$, variando apenas o parâmetro t ao longo do raio.
- Encontrar t^* tal que $g(t^*) = 0$ localiza exatamente onde o raio "toca" a superfície.
- Bisseção é usada na prática em *shaders* para refinar a posição da interseção após uma varredura grosseira detectar troca de sinal.

Problema 4 – Fator de carga em tabelas hash

Enunciado

O tempo médio esperado de busca em uma tabela hash com encadeamento cresce com o fator de carga α segundo $C(\alpha) = 1 + \alpha/2$. Deseja-se saber para qual α o custo médio $C(\alpha)$ atinge um limite operacional de 1,8 acessos.

- Formule como zero de função: $g(\alpha) = C(\alpha) - 1,8$.
- Neste caso simples g é linear e a solução é direta, mas em modelos mais realistas (com colisões e re-hash), $C(\alpha)$ pode ser não linear – exigindo um método numérico.
- Aplicação: decidir em que fator de carga disparar o **redimensionamento** (*rehashing*) da tabela.

Problema 5 – Dimensionamento de servidor (fila M/M/1)

Enunciado

Em um modelo de fila M/M/1, o tempo médio de espera é $W(\rho) = \frac{\rho}{\mu(1-\rho)}$, onde ρ é a utilização do servidor e μ a taxa de atendimento. Encontrar ρ tal que $W(\rho)$ seja igual ao SLA (*Service Level Agreement*) de 200 ms.

- Formule como zero de função: $g(\rho) = W(\rho) - 0,2$, com $\rho \in (0, 1)$.
- $W(\rho) \rightarrow \infty$ quando $\rho \rightarrow 1$, garantindo troca de sinal no intervalo de interesse.
- Aplicação direta em **engenharia de desempenho** de sistemas web, bancos de dados e infraestrutura de nuvem: determinar até que utilização o servidor pode operar respeitando o SLA.

Exemplo resolvido em Python (Problema 5)

```
1 def tempo_espera(rho, mu=50):
2     """W(rho) = rho / (mu * (1 - rho))  -- fila M/M/1."""
3     return rho / (mu * (1 - rho))
4
5 def g(rho):
6     return tempo_espera(rho) - 0.2    # SLA = 200 ms = 0.2 s
7
8 # Intervalo inicial: rho proximo de 0 (fila vazia) ate rho proximo de 1
9 raiz, n_iter = bissecao(g, 0.01, 0.99, tol=1e-6)
10 print(f"utilizacao maxima = {raiz:.4f} (em {n_iter} iteracoes)")
11 # Acima desse rho, o SLA de 200 ms deixa de ser cumprido
```

Onde os alunos de CC da UFERSA encontrarão esses métodos

- **Inteligência Artificial e Aprendizado de Máquina:** calibração de limiares, busca de hiperparâmetros unidimensionais, ajuste de funções de ativação e regularização.
- **Computação Gráfica e Processamento de Imagens:** interseção raio-superfície, contornos implícitos, detecção de bordas por variação de intensidade.
- **Redes de Computadores:** dimensionamento de buffers e filas, cálculo de pontos de saturação de enlace, modelos de tráfego.
- **Engenharia de Software e Desempenho:** análise de complexidade comparada (pontos de cruzamento entre algoritmos), definição de limites operacionais de sistemas.

Onde os alunos de CC da UFERSA encontrarão esses métodos (cont.)

- **Criptografia e Segurança:** localização de parâmetros críticos em modelos de força de senha/entropia, análise de curvas de custo computacional de ataques.
- **Otimização e Pesquisa Operacional** (ligado ao GEMAP): métodos de intervalo aparecem como sub-rotinas em buscas unidimensionais (*line search*) dentro de algoritmos de otimização maiores.
- **Bioinformática e Ciência de Dados:** ajuste de curvas de calibração, definição de pontos de corte (*cutoffs*) em testes estatísticos e modelos preditivos.
- **Simulação Computacional** (aplicações em ciência e tecnologia, como carcinicultura e outros modelos de otimização estudados no grupo de pesquisa): localizar o ponto em que uma variável de simulação cruza um valor crítico.

Por que começar por Bisseção e Falsa Posição?

- São a base conceitual antes de métodos mais rápidos, porém menos robustos (Newton-Raphson, Secante – já estudados nesta disciplina);
- Não exigem derivada, funcionando mesmo quando f vem de uma **simulação** ou de **dados** (sem fórmula fechada);
- Garantia teórica de convergência: essencial em sistemas críticos onde uma falha na convergência não é aceitável;
- Compreender *quando* usar cada método é uma habilidade central de um cientista da computação que trabalha com modelagem numérica.

Método	Regra de atualização	Característica
Bisseção	$x_m = (a + b)/2$	Robusto, previsível, mais lento
Falsa Posição	$x_f = \frac{af(b) - bf(a)}{f(b) - f(a)}$	Usa valor de f , pode estagnar

- Ambos exigem apenas f contínua e troca de sinal em $[a, b]$ – métodos de **intervalo fechado**.
- Ambos garantem convergência, ao contrário de métodos abertos (Newton-Raphson, Secante), que podem divergir dependendo do chute inicial.
- Amplamente aplicáveis em problemas reais de Ciência da Computação sempre que um parâmetro precisa ser calibrado numericamente.

Conclusão

- Bisseção e Falsa Posição são ferramentas fundamentais para encontrar zeros de funções quando não há solução analítica.
- A escolha entre eles envolve um equilíbrio entre **robustez previsível** (Bisseção) e **potencial de convergência mais rápida** (Falsa Posição, com o risco de estagnação).
- Em Ciência da Computação, esses métodos aparecem sempre que é preciso resolver numericamente uma equação que surge de modelos de desempenho, calibração ou geometria computacional.

Referências

RUGGIERO, M. A. G.; LOPES, V. L. R. *Cálculo Numérico: Aspectos Teóricos e Computacionais*. 2ª ed. Pearson, 1997.

BURDEN, R. L.; FAIRES, J. D. *Numerical Analysis*. 10ª ed. Cengage Learning, 2015.

Obrigado!

Prof. Paulo César Linhares da Silva
linhares@ufersa.edu.br